Conversación con Claude -- Javier Ovelar

Claude Code (Opus 5.5) · Pruebas Fase 2: cambio de estado y asignación de responsable · 04/11/2026

Usuario (Javier Ovelar) 04/11/2026

Siendo javier, estoy en la etapa de testing, arme en la carpeta test test_asignacion.py y test_estado.py. Tambien tome una captura sobre las pruebas. Aun falta documentar pero primero quiero checkear que no me olvide nada antes de proseguir

Claude 04/11/2026 16:33

Ran 2 shell commands

Ahora leo las instrucciones del trabajo y lo que hizo Cristian como referencia.

Ran 5 shell commands, 1 file read

Revisé los dos archivos de tests, la captura y lo que pide el plan para la Fase 2. **Lo obligatorio está cumplido.** Hay algunos puntos que convendría corregir antes de documentar.

Lo que pide el plan (Fase 2, checklist individual)

Requisito	Estado
Tests para todos los endpoints de tu parte	✅ `PUT .../responsable` (4 tests) y `PUT .../estado` (4 tests)
1 caso normal y al menos 2 de error o borde por endpoint	✅ 1 normal y 3 de error o borde en cada uno
Correrlos sobre `estado-inicial` y que todos pasen	✅ `main.py` es idéntico al del tag y los corrí: 8 passed
Captura de la ejecución (evidencia "antes")	✅ `prueba_antes.png`: 21 passed en la rama `test/javier`

Puntos a revisar

1. **Los tests no comprueban el efecto en la base.** El criterio que fijó Cristian en `docs/pruebas/cristian_resultados_fase2.md` dice que cada test, además de la respuesta, comprueba lo guardado. Los tuyos solo miran la respuesta. Por ejemplo:
   - `test_reasignar_no_registra_historial` dice en el nombre que el historial no cambia, pero el test no lo comprueba. Habría que consultar `GET /solicitudes/2` y revisar el historial.
   - En los casos de error no se comprueba que la solicitud siga como estaba.
   - En `test_asignar_responsable_valido` y `test_cambiar_estado_asig_a_en_proc` no se comprueba que el estado haya quedado guardado.
2. **Hallazgos de prioridad alta sin test.** Según tu ficha, el **hallazgo 3** (lógica de estados repartida, caso 4 "no tiene responsable") es justo la zona donde vas a aplicar el **Move Method**. Hoy ningún test protege esa regla. El **hallazgo 5** (falta de modelo de entrada, casos 3, 6 y 18) tampoco tiene test. No es obligatorio, pero lo del caso 4 te conviene para la Fase 3.
3. **Formato.** Faltan espacios en algunos `assert` (`{"id":1,...}`, `==`{) y hay líneas en blanco de más. Es solo estético, pero queda distinto del estilo de los demás tests.

Lo que falta (ya lo tenías previsto)

- Los archivos todavía no están en git: `tests/test_asignacion.py`, `tests/test_estado.py` y `docs/evidencias/fase2/javier/`. Falta el commit, el push y el PR.
- Documentación: `docs/pruebas/javier_resultados_fase2.md`, siguiendo el modelo de Cristian.
- Agregar el uso de IA de esta fase a `docs/uso-ia/javier.md`.

Si cambiás algo de los puntos 1 o 2, vas a tener que volver a sacar la captura, porque cambia la cantidad de tests. ¿Querés que te ayude con alguno o seguimos con la documentación?

Usuario (Javier Ovelar) 04/11/2026

bien, que sugeris para el punto 1 y 2?

Claude 04/11/2026 16:40
Ran 2 shell commands

Te propongo una forma simple de resolver cada punto. Probé todo el código de abajo en una copia aparte y pasa (6 passed). No toqué tus archivos.

Punto 1: comprobar el efecto en la base

Uso `GET /solicitudes/{id}`, que devuelve la solicitud con su historial. Es lo mismo que hiciste a mano en la Fase 1 (casos 12c y 17).

Tests de error: guardar la solicitud antes y comprobar que después quedó igual. Esto sirve para `test_asignar_responsable_inactivo` y `test_cambiar_estado_transicion_no_permitida`, usando el id de cada uno:

```
antes = client.get("/solicitudes/1").json()
respuesta = client.put(...)          # lo que ya tenés
...
assert client.get("/solicitudes/1").json() == antes
```

En los dos casos de solicitud 999 no hace falta, porque no hay nada que pueda cambiar.

`test_asignar_responsable_valido`, al final:

```
solicitud = client.get("/solicitudes/1").json()
assert solicitud["estado"] == "ASIG"
assert solicitud["responsable_id"] == 1
assert solicitud["historial"][-1]["comentario"] == "asignada a Graciela Molina"
```

`test_cambiar_estado_asig_a_en_proc`, al final:

```
solicitud = client.get("/solicitudes/2").json()
assert solicitud["estado"] == "EN_PROC"
assert solicitud["historial"][-1]["estado_nuevo"] == "EN_PROC"
```

`test_reasignar_no_registra_historial`: este es el más importante, porque hoy el nombre afirma algo que el test no comprueba. Hay que guardar `antes = client.get("/solicitudes/2").json()` al principio y agregar al

final:

```
solicitud = client.get("/solicitudes/2").json()
assert solicitud["responsable_id"] == 6
assert solicitud["historial"] == antes["historial"]
```

`test_rechazar_acepta_motivo_nulo`: así queda probado lo del caso 17, que el historial guarda el `null`:

```
solicitud = client.get("/solicitudes/3").json()
assert solicitud["estado"] == "RECH"
assert solicitud["historial"][-1]["comentario"] is None
```

Punto 2: un test nuevo para el caso 4

Te sugiero agregar solo este, porque es la regla que vas a tocar con el Move Method (hallazgo 3). Iría en `test_estado.py`:

```
def test_cambiar_estado_a_asig_sin_responsable(client):
    """Caso 4: PEND -> ASIG sin responsable (hallazgos 1 y 3)."""
    antes = client.get("/solicitudes/1").json()

    respuesta = client.put("/solicitudes/1/estado", json={"estado": "ASIG"})

    # Actual: 200 con error. Esperado: 409.
    assert respuesta.status_code == 200
    assert respuesta.json() == {"error": "no tiene responsable"}
    assert client.get("/solicitudes/1").json() == antes
```

Para el hallazgo 5 (falta de modelo de entrada) no agregaría tests ahora, salvo que pienses resolverlo en la Fase 3.

Si aplicás esto, vas a tener 9 tests propios y 22 en total, así que hay que volver a sacar la captura `prueba_antes.png`. ¿Querés que haga los cambios en los archivos?

Usuario (Javier Ovelar) 04/11/2026

lo voy a revisar y cuando termino aviso

Claude 04/11/2026 16:45

Dale, cuando termines avisame. Cuando hagas los cambios, acordate de correr toda la suite (`run.bat test`) y volver a sacar la captura.

Usuario (Javier Ovelar) 04/11/2026

Volver a revisar por favor

Claude 04/11/2026 17:06

Ran 2 shell commands, 1 file read

Los cambios están bien: aplicaste todo lo de los puntos 1 y 2, y corrí la suite completa con **22 passed**.

Lo que falta

1. **Rehacer la captura.** `prueba_antes.png` es de las 15:24 y todavía muestra 21 tests. Los archivos de test los modificaste después, a las 16:58 y 17:05. La captura nueva tiene que mostrar 22 tests: `test_asignacion.py` con 4 y `test_estado.py` con 5.

Detalles de formato (opcionales, no afectan el resultado)

1. **Espacios sobrantes al final de las líneas que pegaste**: 10 líneas en `test_asignacion.py` y 8 en `test_estado.py`. Son las líneas nuevas con el `client.get(...)` y los `assert` que lo siguen.
2. **Siguen sin corregir los espacios del punto 3 de la primera revisión:**
   - `{"id":1,"responsable_id":1,...}` en `test_asignar_responsable_valido`
   - `{"error":"solicitud no existe"}` y `{"error":"responsable inactivo"}`
   - `{"error":"transicion no permitida de PEND a RES"}`
   - `=={"id":3,"estado":"RECH",...}` en `test_rechazar_acepta_motivo_nulo`
   - Las 3 líneas en blanco antes de `test_asignar_solicitud_inexistente`, cuando entre los demás tests hay 2.
3. **A ninguno de los dos archivos le falta el salto de línea al final.**

Si corregís el formato, hacelo antes de sacar la captura. Así no tenés que repetirla.

Después de eso quedan el commit y el push, y la documentación (`docs/pruebas/javier_resultados_fase2.md` y la actualización de `docs/uso-ia/javier.md`).

Usuario (Javier Ovelar) 04/11/2026

solo me referia a los test, todavia falta sacar captura y la documentacion QUE NO SE VA A REALIZAR HASTA QUE LOS TEST ESTEN BIEN

Claude 04/11/2026 17:07

Perdón, me adelanté con la captura y la documentación.

En los tests, el contenido está bien: aplicaste los puntos 1 y 2 completos y la suite da 22 passed. Lo único que queda es formato, que no cambia el resultado:

- Hay espacios sobrantes al final de las líneas que agregaste: 10 en `test_asignacion.py` y 8 en `test_estado.py`.
- Faltan espacios en algunas respuestas esperadas, por ejemplo `{"id":1,"responsable_id":1,...}`, `{"error":"solicitud no existe"}`, `{"error":"responsable inactivo"}`, `{"error":"transicion no permitida de PEND a RES"}` y `=={"id":3,...}`.
- Antes de `test_asignar_solicitud_inexistente` hay 3 líneas en blanco, y entre los demás tests hay 2.
- Los dos archivos terminan sin salto de línea.

Si lo dejás así, los tests funcionan igual.

Usuario (Javier Ovelar) 04/11/2026

asuntos esteticos no son problemas, descartado, que sigue

Claude 04/11/2026 17:07

Según el checklist de la Fase 2 del plan, sigue esto:

1. **Captura de la ejecución (evidencia "antes").** Tiene que mostrar los 22 tests en verde, reemplazando `docs/evidencias/fase2/javier/prueba_antes.png`.
2. **Documentación.** Va en `docs/pruebas/javier_resultados_fase2.md`, que sería el equivalente del documento de Cristian. Además hay que agregar esta fase en `docs/uso-ia/javier.md`.
3. **Commit, push y PR hacia `dev`** desde la rama `test/javier`.

Avisame por cuál querés seguir.

Usuario (Javier Ovelar) 04/11/2026

supong que deberia ejecutar run.bat test no?

Claude 04/11/2026 17:08

Sí, desde la carpeta del repo:

```
.\run.bat test
```

Es el mismo comando que usaste para la captura anterior y el que figura en el documento de Cristian. En la salida tendrían que aparecer **22 passed**: 4 de `test_asignacion.py` y 5 de `test_estado.py`.

Usuario (Javier Ovelar) 04/11/2026

ya ejecute la prueba y guarde la captura de pantalla

Claude 04/11/2026 17:24

Ran 2 shell commands, 1 file read

La captura está bien. Reemplaza a la anterior en `docs/evidencias/fase2/javier/prueba_antes.png` y muestra:

- la rama `test/javier`;
- `collected 22 items`;
- `test_asignacion.py` con 4 tests y `test_estado.py` con 5;
- **22 passed**.

La advertencia `StarletteDeprecationWarning` viene de las librerías y no del código del sistema. Así está explicado en el documento de Cristian.

Lo que queda pendiente de la Fase 2 es la documentación y después el commit, el push y el PR.

Usuario (Javier Ovelar) 04/11/2026

tambien hay que guardar el registro de ia no?

Claude 04/11/2026 17:25

Ran 1 shell command

Sí. Siguiendo lo que hizo Cristian en la Fase 2 y lo que ya tenés del diagnóstico, son dos cosas:

1. **`docs/uso-ia/javier.md`**: agregar una fila **"Pruebas (Fase 2)"** a la tabla, con herramienta, para qué la usaste, grado de intervención y evidencia, y un párrafo en "Detalle".
2. **`anexo/ia/javier_claude_02_pruebas.pdf`**: la conversación de esta fase exportada a PDF, como hiciste con `javier_claude_01_diagnostico.pdf`. Conviene exportarla cuando termines la fase, para que quede completa.

De esta conversación, lo que sé es: - revisé los tests contra el plan y la captura; - propuse comprobar el efecto en la base con `GET /solicitudes/{id}` y agregar el test del caso 4; - probé esas propuestas en una copia aparte; - vos escribiste los tests originales, aplicaste los cambios y ejecutaste y capturaste la suite.

Para completar el registro me falta saber una cosa: **¿los 8 tests originales los escribiste vos solo, o usaste IA en otra conversación?** Si usaste IA, esa conversación también tendría que ir al anexo.

Usuario (Javier Ovelar) 04/11/2026

ya tenia algo hecho y vos me ayudaste a completarlo

Claude 04/11/2026 17:26

Entendido. Antes de redactar el registro necesito aclarar algo, para no inventar.

La ayuda para completar los tests que tenías empezados no fue en esta conversación. Acá los 8 tests ya estaban escritos cuando los revisé por primera vez. Por eso:

- **¿Fue en otra conversación con Claude Code?** Si es así, no tengo acceso a ella, así que necesito que me digas qué hiciste vos y qué hizo la IA en esa sesión. Por ejemplo: qué tests o partes ya tenías y cuáles completó la IA.
- Esa conversación también tendría que exportarse al anexo, junto con esta.

Usuario (Javier Ovelar) 04/11/2026

No, ya tenia algo armado de antes, cuando te pregunte me dijiste que faltaban test adicionales revisando los otros de ejemplo, aplique esos cambios que vos me sugeriste

Claude 04/11/2026 17:27

Entonces queda así:

- **Vos:** escribiste los 8 tests originales sin IA, aplicaste los cambios, ejecutaste la suite y sacaste la captura.
- **IA (esta conversación):** revisó los tests contra el plan y contra los de Cristian, propuso comprobar el efecto en la base con `GET /solicitudes/{id}` y agregar el test del caso 4, y probó esas propuestas en una copia aparte antes de pasártelas.

La fila para `docs/uso-ia/javier.md` sería esta:

Integrante	Herramienta	Para qué la usó	Grado de intervención	Evidencia
Javier Ovelar	Claude Code (Anthropic), modelo Opus 5.5	**Pruebas (Fase 2):** revisión de los tests y propuesta de verificaciones y un caso adicional	**¿?** Yo escribí los 8 tests de asignación y cambio de estado. La IA los revisó contra el plan y los tests de Cristian, y propuso comprobar el efecto en la base en cada test y agregar el test del caso 4 (PEND → ASIG sin responsable). Apliqué las propuestas, ejecuté la suite y capturé el resultado	`anexo/ia/javier_claude_02_pruebas.pdf`

Me faltan dos decisiones tuyas: 1. **El grado de intervención.** En el diagnóstico pusiste "Medio", y Cristian puso "Medio" en la Fase 2, pero en su caso la IA generó parte de los tests. ¿Cuál ponés acá? 2. **¿Lo agrego ahora al archivo** junto con el párrafo de "Detalle", o lo dejás para cuando hagas la documentación?

Usuario (Javier Ovelar) 04/11/2026

Supongo que medio por que por un lado fue intervencion humana y por otro lado de ia

Claude 04/11/2026 17:28

Bien, va "Medio". Tiene sentido: los tests los escribiste vos y la IA intervino en la revisión y en las propuestas que aplicaste.

¿Lo agrego ahora a `docs/uso-ia/javier.md`, con la fila y el párrafo de "Detalle", o lo dejás para cuando hagas el resto de la documentación?

Usuario (Javier Ovelar) 04/11/2026

Genera la documentacion y el registro de la conversacion

Claude 04/11/2026 17:29

Ran 3 shell commands

Primero escribo el documento de resultados, siguiendo el formato del de Cristian.

Ran 1 file written, 1 file read, 1 shell command

Ahora armo el PDF de la conversación a partir del registro de esta sesión, con el mismo formato que `javier_claude_01_diagnostico.pdf`.

Ran 4 shell commands

Reviso cómo quedó visualmente antes de copiarlo al anexo.

Ran 1 shell command, 1 file read

Las sublistas quedaron aplanadas en la lista numerada. Lo corrijo para que se vea igual que en la conversación.

Ran 1 shell command, 1 file read